

Real-Time Supply Chain Risk Analysis Using Apache Kafka and Apache Spark Structured Streaming

Technical Report

Big Data Analytics — System Design, Implementation & Evaluation

Members: 16014223052 Mit Mhatre
16014223041 Jeet Patil

Batch: B1

Technology Stack: Apache Kafka 3.9.2, Apache Spark 3.3.4

Language: Python 3.x

ML Framework: scikit-learn + joblib

Dataset: DataCo Global Supply Chain Dataset

Platform: Windows 10/11, JDK 1.8

1. Introduction and Motivation

In modern global commerce, supply chain disruptions — delayed shipments, unprofitable orders, suboptimal routing — can cost organisations millions of dollars annually. Traditional batch-processing analytics systems are ill-suited for responding to such issues in real time because they process data hours or days after events occur. The need for low-latency, event-driven decision-making has driven adoption of streaming architectures that can ingest, process, and act on data within seconds of its arrival.

This project addresses that gap by building a **real-time supply chain risk analysis pipeline** that combines two industry-standard open-source technologies:

- **Apache Kafka** — a distributed, fault-tolerant message broker used for decoupling data producers from consumers and guaranteeing ordered, durable message delivery.
- **Apache Spark Structured Streaming** — a high-level streaming API built on Spark SQL that treats live data streams as continuously appended DataFrames, enabling SQL-style transformations on real-time data.

The system ingests order records from the DataCo Global Supply Chain dataset, streams them through a Kafka topic one record per second (simulating a live order-entry system), applies a pre-trained scikit-learn model to predict shipment delay, and then runs a rule-based decision engine that assigns a composite risk score and a business-level action recommendation.

1.1 Objectives

1. Design and implement a modular, production-inspired streaming pipeline using Kafka and Spark.
2. Integrate a serialised scikit-learn model into a Spark UDF for real-time inference without retraining overhead.
3. Develop a quantitative risk scoring mechanism combining delay and profit signals.
4. Translate risk scores into actionable business decisions using a priority-ordered rule engine.
5. Validate the pipeline end-to-end with live console output on real supply chain data.

1.2 Document Structure

Section 2 describes the overall system architecture and technology stack. Sections 3 through 9 detail each software module. Section 10 covers the proof-of-concept log-alerting sub-pipeline. Section 11 documents the deployment procedure. Section 12 presents and analyses real sample outputs from the running system. Section 13 concludes the report with a critical evaluation and future directions.

2. System Architecture

2.1 High-Level Overview

The architecture follows a **Kappa architecture** pattern — a simplification of the Lambda architecture that uses a single streaming path for both real-time and historical data processing. By replaying historical CSV records through Kafka at a controlled rate, the system treats historical data identically to live order data, avoiding a separate batch-processing layer.

The end-to-end data flow is:

```

CSV (DataCo)  $\xrightarrow{\text{producer.py}}$  Kafka: supply_chain  $\xrightarrow{\text{readStream}}$  JSON parse  $\rightarrow$  Feature Eng.
 $\rightarrow$  ML Predict  $\rightarrow$  Risk Score  $\rightarrow$  Decision  $\rightarrow$  Console

```

2.2 Technology Stack

Table 1: Technology stack and version details

Component	Version	Role
Apache Kafka	3.9.2 (Scala 2.12)	Message broker and topic management
Apache ZooKeeper	Bundled with Kafka	Kafka cluster coordination
Apache Spark	3.3.4 (Hadoop 3)	Distributed stream processing engine
Spark-Kafka connector	0-10.2.12:3.3.4	Bridge between Kafka and Spark SQL
Python	3.x	Application language
kafka-python	Latest	Kafka producer/consumer client
PySpark	3.3.4	Python API for Apache Spark
scikit-learn	Latest	ML model training and inference
joblib	Latest	Model serialisation and loading
pandas	Latest	CSV reading and schema inference
Java JDK	1.8.0_481	JVM runtime for Spark

2.3 Module Inventory

Table 2: Source-file inventory with layer classification

File	Layer	Responsibility
<code>config.py</code>	Config	Central Kafka server URL and topic constants
<code>producer.py</code>	Ingestion	Reads CSV and publishes JSON rows to Kafka
<code>schema.py</code>	Ingestion	Auto-generates Spark StructType from pandas dtypes
<code>spark_consumer.py</code>	Streaming	SparkSession, stream read, pipeline orchestration
<code>feature_engineering.py</code>	ML Prep	Derives <code>shipping_efficiency</code> feature
<code>prediction.py</code>	ML	scikit-learn UDF for shipment delay prediction
<code>decision_engine.py</code>	Business	Risk scoring and order decision logic
<code>main.py</code>	Entry point	Application entry — calls <code>start_stream()</code>
<code>producer_kafka.py</code>	PoC	Random log producer for test topic
<code>consumer_kafka.py</code>	PoC	Simple log consumer filtering errors/warnings
<code>spark_stream.py</code>	PoC	Spark filter on ERROR/WARNING log events

3. Configuration Management (`config.py`)

```

1 KAFKA_SERVER = "localhost:9092"
2 TOPIC        = "supply_chain"

```

Listing 1: `config.py` — centralised Kafka configuration

Centralising connection parameters in a dedicated configuration module follows the **12-Factor App** methodology’s principle of storing configuration separately from code. In a production system, these values would be injected from environment variables or a secrets manager. Benefits include:

- **Single point of change:** migrating the Kafka broker requires editing only one file.
- **Testability:** unit tests can monkey-patch `config.py` to point at a local test broker without modifying application code.
- **Readability:** downstream modules use `from config import KAFKA_SERVER` rather than scattering literal strings throughout the codebase.

4. Data Ingestion Layer

4.1 Kafka Producer (producer.py)

The producer reads the DataCo Supply Chain CSV and publishes each row to the Kafka `supply_chain` topic as a JSON-serialised message.

```
1 from kafka import KafkaProducer
2 import json, pandas as pd, time
3
4 df = pd.read_csv(
5     "DataCoSupplyChainDataset.csv",
6     encoding='latin-1'
7 )
8
9 producer = KafkaProducer(
10     bootstrap_servers='localhost:9092',
11     value_serializer=lambda x: json.dumps(x).encode('utf-8')
12 )
13
14 for _, row in df.head(10000).iterrows():
15     producer.send("supply_chain", row.to_dict())
16     print("Sent one row")
17     time.sleep(1)
```

Listing 2: producer.py — full implementation

Each row is converted to a Python dictionary via `to_dict()` and serialised to UTF-8-encoded JSON bytes using the `value_serializer` lambda. JSON was chosen over binary formats (Avro, Protobuf) because it is human-readable, schema-flexible, and natively supported by Spark's `from_json` function without requiring a schema registry. The `time.sleep(1)` delay throttles output to one message per second, simulating a live transactional system. The 10 000-row cap provides roughly 2.8 hours of simulated runtime for functional testing.

4.2 Schema Auto-Generation (schema.py)

Rather than hand-coding a 50-column `StructType`, the schema is derived programmatically from pandas dtype inference.

```
1 from pyspark.sql.types import *
2 import pandas as pd, joblib
3
4 feature_cols = joblib.load("feature_columns.pkl")
5 df_sample = pd.read_csv(
6     "DataCoSupplyChainDataset.csv", encoding='latin-1'
7 )
8
9 def generate_schema(df):
10     schema = StructType()
11     for col, dtype in zip(df.columns, df.dtypes):
12         if dtype == 'int64': schema.add(col, IntegerType())
13         elif dtype == 'float64': schema.add(col, DoubleType())
14         else: schema.add(col, StringType())
15     return schema
16
```

```
17 schema = generate_schema(df_sample)
```

Listing 3: schema.py — dynamic StructType generation

The function maps three pandas dtype classes to Spark equivalents: `int64` $\rightarrow$ `IntegerType`, `float64` $\rightarrow$ `DoubleType`, and all others $\rightarrow$ `StringType`. The exported module-level `schema` variable is imported directly by `spark_consumer.py`.

5. Spark Structured Streaming (spark_consumer.py)

5.1 SparkSession Initialisation

```
1 spark = SparkSession.builder \
2     .appName("SupplyChainStreaming") \
3     .config(
4         "spark.jars.packages",
5         "org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.4"
6     ) \
7     .getOrCreate()
8
9 spark.sparkContext.setLogLevel("WARN")
```

Listing 4: Creating the SparkSession with Kafka connector

The `spark.jars.packages` configuration tells Spark to download the Kafka SQL connector from Maven Central at startup. The connector version must exactly match both the Spark version (3.3.4) and the Scala version (2.12) used to compile the Spark distribution. Setting the log level to `WARN` suppresses verbose INFO output so that supply chain analysis output is clearly visible in the console.

5.2 Kafka Stream Subscription

```
1 df = spark.readStream \
2     .format("kafka") \
3     .option("kafka.bootstrap.servers", "localhost:9092") \
4     .option("subscribe", "supply_chain") \
5     .option("startingOffsets", "latest") \
6     .load()
```

Listing 5: Subscribing to the supply_chain topic

The `startingOffsets="latest"` setting instructs Spark to consume only messages published after the job starts. For full historical replay after a model update, this would be changed to `"earliest"`.

5.3 Message Deserialisation

Kafka delivers messages as raw byte arrays. Only the `value` column is relevant here.

```
1 df_parsed = df.selectExpr("CAST(value AS STRING)") \
2     .select(from_json(col("value"), schema).alias("data")) \
3     .select("data.*")
```

Listing 6: Deserialising the JSON payload into typed columns

The three-step transformation: (1) casts bytes to UTF-8 string, (2) applies `from_json` with the pre-generated schema to produce a nested struct column, (3) flattens the struct to top-level columns. After this step, `df_parsed` has the same column structure as the original CSV.

5.4 Pipeline Orchestration

```

1 df_features = add_features(df_parsed)
2 df_pred     = apply_prediction(df_features)
3 df_final    = apply_decision(df_pred)
4
5 query = df_final.writeStream \
6     .foreachBatch(format_output) \
7     .outputMode("append") \
8     .start()
9
10 query.awaitTermination()

```

Listing 7: Full pipeline composition and stream start

Each transformation returns a new DataFrame with additional columns, following an immutable transformation pattern consistent with Spark’s lazy evaluation model. `foreachBatch` triggers `format_output` for every micro-batch, giving access to a full Spark DataFrame per interval.

6. Feature Engineering (feature_engineering.py)

```

1 from pyspark.sql.functions import col
2
3 def add_features(df):
4     return df.withColumn(
5         "shipping_efficiency",
6         col("Days for shipment (scheduled)") /
7         (col("Days for shipment (scheduled)") + 1)
8     )

```

Listing 8: feature_engineering.py — shipping efficiency

The shipping efficiency e is defined as:

$$e = \frac{d}{d+1}, \quad d \geq 0$$

where d is the number of scheduled shipping days. Key properties: $e \in (0, 1)$ for all valid $d > 0$ (bounded, no further normalisation required); as $d \rightarrow \infty$, $e \rightarrow 1$; and the formula avoids discontinuities introduced by min-max scaling, which would require knowledge of the maximum observed d at inference time.

Additional features that could be appended include discount ratio, quantity-to-sales ratio, and customer segment priority flags.

7. Machine Learning Prediction (prediction.py)

7.1 Model Artefacts

Four serialised objects are loaded at module initialisation:

Table 3: Serialised model artefacts

Artefact	Description
shipping_delay_model.pkl	Trained regressor predicting delay in days
feature_columns.pkl	Ordered list of column names expected by the model
label_encoders.pkl	Per-column LabelEncoder instances for categoricals
scaler.pkl	StandardScaler fitted on the training set

7.2 UDF Implementation

scikit-learn models cannot operate directly on Spark DataFrames; the prediction logic is wrapped in a Python UDF registered with Spark's Catalyst optimiser.

```

1 from pyspark.sql.functions import udf, struct, col
2 from pyspark.sql.types import DoubleType
3 import joblib, warnings
4 warnings.filterwarnings("ignore",
5     message="X does not have valid feature names")
6
7 model      = joblib.load("shipping_delay_model.pkl")
8 feature_cols = joblib.load("feature_columns.pkl")
9 label_enc  = joblib.load("label_encoders.pkl")
10 scaler     = joblib.load("scaler.pkl")
11
12 def predict_delay(row):
13     try:
14         values = []
15         for c in feature_cols:
16             v = row[c]
17             if c in label_enc:
18                 try:
19                     v = int(label_enc[c].transform([str(v)])[0])
20                 except Exception:
21                     v = -1 # unseen category -> sentinel
22             else:
23                 try: v = float(v) if v is not None else 0.0
24                 except: v = 0.0
25             values.append(v)
26         scaled = scaler.transform([values])
27         return float(model.predict(scaled)[0])
28     except Exception as e:
29         print("Prediction error:", e)
30         return None
31
32 delay_udf = udf(predict_delay, DoubleType())

```

Listing 9: prediction.py — scikit-learn UDF

The UDF replicates the full preprocessing pipeline per row: feature extraction in the exact order defined by `feature_cols.pkl`, label encoding with sentinel `-1` for unseen categories, type coercion (`None` $\rightarrow$ `0.0`), standard scaling, and model inference.

7.3 Applying Predictions

```

1 def apply_prediction(df):
2     return df \
3         .withColumn(
4             "pred_delay",
5             delay_udf(struct(*feature_cols))
6         ) \
7         .withColumn(
8             "pred_profit",
9             col("Order Profit Per Order").cast(DoubleType())
10        )

```

Listing 10: `apply_prediction` — attaching `pred_delay` and `pred_profit`

The `struct(*feature_cols)` call packs the required columns into a single struct value passed to the UDF — the standard pattern for multi-column Spark UDFs. The existing profit column is cast to `DoubleType` and aliased as `pred_profit`.

8. Decision Engine (`decision_engine.py`)

8.1 Delay Cleaning and Risk Score

```

1 from pyspark.sql.functions import when, col, round as spark_round
2
3 def apply_decision(df):
4     return df \
5         .withColumn("clean_delay",
6                     spark_round(when(col("pred_delay") < 0, 0)
7                                   .otherwise(col("pred_delay")), 2)) \
8         .withColumn("delay_score",
9                     when(col("clean_delay") > 5, 1.0)
10                    .otherwise(col("clean_delay") / 5)) \
11         .withColumn("profit_flag",
12                     when(col("pred_profit") < 0, 1).otherwise(0)) \
13         .withColumn("risk_score",
14                     spark_round(
15                         (col("delay_score") * 60) +
16                         (col("profit_flag") * 40), 2)) \
17         .withColumn("decision",
18                     when((col("clean_delay") > 3) & (col("pred_profit") < 0),
19                          "Reject Order")
20                    .when(col("clean_delay") > 2, "Change Shipping Mode")
21                    .when(col("pred_profit") < 0, "Reduce Discount")
22                    .otherwise("Accept Order"))

```

Listing 11: `decision_engine.py` — risk scoring and decision

Negative delay predictions are floored to zero. The composite risk score is:

$$R = 60 \times \min\left(\frac{\text{cleanDelay}}{5}, 1.0\right) + 40 \times \text{ProfitFlag}$$

The 60/40 weighting reflects the assumption that delivery delay has broader operational impact than a marginally unprofitable order.

8.2 Business Decision Rule Table

Table 4: Decision rules evaluated top-to-bottom (first match wins)

Condition	Decision	Rationale
Delay > 3 days AND Profit < 0	Reject Order	Both slow and unprofitable; cost exceeds benefit
Delay > 2 days	Change Shipping Mode	Upgrade shipping to mitigate delay risk
Profit < 0	Reduce Discount	Margin issue; renegotiate before processing
Otherwise	Accept Order	Within acceptable risk parameters

9. Output Formatting and Console Display

```

1 def format_output(df, epoch_id):
2     rows = df.collect()
3     for row in rows:
4         print("\n==== SUPPLY CHAIN ANALYSIS =====")
5         print(f"Sales: {row['Sales per customer']}")
6         print(f"Quantity: {row['Order Item Quantity']}")
7         print(f"Category: {row['Category Name']}")
8         print(f"Customer Segment: {row['Customer Segment']}")
9         print("\n----- PREDICTION -----")
10        print(f"Predicted Delay: {row['pred_delay']:.2f} days")
11        print(f"Predicted Profit: ${row['pred_profit']:.2f}")
12        print("\n----- RISK ANALYSIS -----")
13        print(f"Risk Score: {row['risk_score']}")
14        print("\n----- FINAL DECISION -----")
15        print(f"Decision: {row['decision']}")
16        print("\n===== \n")

```

Listing 12: spark_consumer.py — per-batch output formatter

`df.collect()` materialises all rows from distributed workers to the driver node for printing. This is acceptable for low-throughput demonstrations; in production, output would target a durable sink such as a database or downstream Kafka topic.

10. Proof-of-Concept Log Alerting Pipeline

Before building the main supply chain pipeline, a simpler Kafka log-alerting system was implemented to validate Kafka-Spark connectivity.

`producer_kafka.py` randomly sends one of three log strings (INFO: OK, WARNING: CPU High, ERROR: Disk Fail) to a `test-topic` each second. Two consumers process these messages:

```

1 from kafka import KafkaConsumer
2
3 consumer = KafkaConsumer(
4     'test-topic',
5     bootstrap_servers='localhost:9092',
6     auto_offset_reset='earliest',
7     group_id='group1'
8 )
9
10 for message in consumer:
11     msg = message.value.decode('utf-8')
12     if "ERROR" in msg or "WARNING" in msg:
13         print(f"ALERT: {msg}")

```

Listing 13: consumer_kafka.py — plain Python log alerter

```

1 df = spark.readStream \
2     .format("kafka") \
3     .option("kafka.bootstrap.servers", "localhost:9092") \
4     .option("subscribe", "test-topic") \
5     .load()
6
7 filtered = df.selectExpr("CAST(value AS STRING)") \
8     .filter(col("value").contains("ERROR") |
9           col("value").contains("WARNING"))
10
11 query = filtered.writeStream \
12     .outputMode("append") \
13     .format("console") \
14     .option("checkpointLocation", "C:/kafka-project/checkpoint") \
15     .start()

```

Listing 14: spark_stream.py — Spark-based log filter with checkpointing

This PoC also demonstrates a **checkpoint directory** — a fault-tolerance mechanism absent from the main pipeline that should be added in future work.

11. Deployment and Startup Procedure

11.1 Prerequisites

- Windows 10/11, PowerShell 5.1+, Java JDK 1.8.0_481
- Kafka 3.9.2 at C:\kafka\kafka_2.12-3.9.2
- Spark 3.3.4 at C:\spark\spark-3.3.4-bin-hadoop3
- Python environment: kafka-python, pyspark, scikit-learn, joblib, pandas
- Model .pkl artefacts and DataCo CSV in their expected paths

11.2 Step-by-Step Startup

Step 1 — Start ZooKeeper

```

cd C:\kafka\kafka_2.12-3.9.2
.\bin\windows\zookeeper-server-start.bat .\config\zookeeper.properties

```

Step 2 — Start Kafka Broker

```
.\bin\windows\kafka-server-start.bat .\config\server.properties
```

Step 3 — Create and Verify Topic

```
.\bin\windows\kafka-topics.bat --create --topic supply_chain \  
    --bootstrap-server localhost:9092 \  
    --partitions 1 --replication-factor 1  
  
.\bin\windows\kafka-topics.bat --list --bootstrap-server localhost:9092
```

Step 4 — Set Environment and Run

```
$env:JAVA_HOME = "C:\Java\jdk1.8.0_481"  
$env:SPARK_HOME = "C:\spark\spark-3.3.4-bin-hadoop3"  
$env:PATH = "$env:SPARK_HOME\bin;$env:JAVA_HOME\bin;$env:PATH"  
  
python producer.py      # Terminal 1  
python main.py          # Terminal 2  
  
# To stop: taskkill /F /IM java.exe
```

12. System Output: Live Results and Analysis

The following screenshots were captured from the running pipeline and demonstrate the system producing correct, interpretable results across three distinct real supply chain scenarios.

12.1 Sample Output 1 — Reduce Discount Decision

```
===== SUPPLY CHAIN ANALYSIS =====
Sales: 63.75
Quantity: 3
Category: Hockey
Customer Segment: Consumer

----- PREDICTION -----
Predicted Delay: 1.47 days
Predicted Profit: $-18.04

----- RISK ANALYSIS -----
Risk Score: 57.64

----- FINAL DECISION -----
Decision: Reduce Discount

=====
```

Figure 1: Live pipeline output — Hockey / Consumer: Reduce Discount decision

Analysis: Sample 1

This Hockey order for a Consumer customer has sales of \$63.75 and a quantity of 3. The model predicts a modest delay of **1.47 days** — below the 2-day shipping escalation threshold. However, the predicted profit of **-\$18.04** is negative, triggering the profit-loss rule. The delay score contributes $1.47/5 \times 60 = 17.64$ points; the profit flag contributes $1 \times 40 = 40$ points, yielding a risk score of **57.64**. The system correctly issues a **Reduce Discount** recommendation, indicating the product is being sold at a margin-eroding discount that should be revised before processing.

12.2 Sample Output 2 — Change Shipping Mode Decision

```
===== SUPPLY CHAIN ANALYSIS =====  
Sales: 48.5  
Quantity: 2  
Category: Hockey  
Customer Segment: Consumer  
  
----- PREDICTION -----  
Predicted Delay: 2.21 days  
Predicted Profit: $-1.41  
  
----- RISK ANALYSIS -----  
Risk Score: 66.52  
  
----- FINAL DECISION -----  
Decision: Change Shipping Mode  
  
=====
```

Figure 2: Live pipeline output — Hockey / Consumer: Change Shipping Mode decision

Analysis: Sample 2

This order has the same category and segment as Sample 1 but lower sales (\$48.5) and a higher predicted delay of **2.21 days** — just above the 2-day threshold triggering the shipping escalation rule. Although the profit is also slightly negative (-\$1.41), the delay rule takes priority in the evaluation chain. The risk score of **66.52** reflects the delay contribution ($2.21/5 \times 60 = 26.52$) plus the profit flag (40). The decision **Change Shipping Mode** is correct: upgrading to express or priority shipping can bring the expected delay within acceptable limits.

12.3 Sample Output 3 — Accept Order Decision

```
===== SUPPLY CHAIN ANALYSIS =====
Sales: 48.5
Quantity: 2
Category: Hockey
Customer Segment: Home Office

----- PREDICTION -----
Predicted Delay: -1.09 days
Predicted Profit: $13.58

----- RISK ANALYSIS -----
Risk Score: 0.0

----- FINAL DECISION -----
Decision: Accept Order

=====
```

Figure 3: Live pipeline output — Hockey / Home Office: Accept Order decision

Analysis: Sample 3

This Home Office customer order is a clear low-risk scenario. The model predicts a **negative delay of -1.09 days** (early delivery), floored to 0 by `clean_delay`. The predicted profit is a healthy **+\$13.58**, so the profit flag is 0. Both components contribute nothing to the risk formula, producing a risk score of exactly **0.0** — the theoretical minimum. The decision **Accept Order** confirms the order should proceed without intervention, validating that the floor logic for negative delays and the zero-risk path through the decision engine both function correctly.

12.4 Cross-Output Comparison

Table 5: Side-by-side comparison of three live pipeline outputs

Attribute	Sample 1	Sample 2	Sample 3
Sales (\$)	63.75	48.50	48.50
Quantity	3	2	2
Customer Segment	Consumer	Consumer	Home Office
Predicted Delay (days)	1.47	2.21	-1.09 ($\rightarrow$ 0)
Predicted Profit (\$)	-18.04	-1.41	+13.58
Risk Score	57.64	66.52	0.0
Decision	Reduce Discount	Change Shipping Mode	Accept Order

The three outputs collectively exercise all decision paths except Reject Order, validating the correctness of the rule engine across a range of delay and profit combinations.

13. Conclusion

13.1 Summary of Achievements

This project successfully demonstrates a complete, working real-time supply chain risk analysis system built on industry-standard open-source big data components. Key achievements include:

- **End-to-end streaming pipeline:** Data flows from a CSV file through Apache Kafka, into Spark Structured Streaming, through an ML model, and out as a business decision within seconds of a record being published.
- **ML integration in a streaming context:** The successful wrapping of a scikit-learn model as a Spark UDF bridges the Python ML ecosystem with JVM-based distributed processing, a non-trivial integration challenge.
- **Correct decision logic:** As demonstrated by the three sample outputs, the decision engine handles all decision paths correctly, including edge cases such as negative predicted delay and zero-risk orders.
- **Modular, maintainable architecture:** Config, ingestion, streaming, feature engineering, prediction, and decision layers are cleanly separated, each independently testable and replaceable.
- **Validated dual pipeline:** The simpler log-alerting PoC pipeline validated infrastructure connectivity before the more complex ML pipeline was introduced.

13.2 Limitations

- The main pipeline lacks a persistent **checkpoint**, so a crash requires manual offset management or topic replay.
- Hard-coded file paths in `prediction.py` and `schema.py` make the application non-portable without modification.

- The row-level Python UDF does not benefit from vectorised execution, limiting throughput at high message rates.
- Output is written to the console only — no durable storage of decisions is implemented.
- A single Kafka partition limits horizontal scalability.

13.3 Future Enhancements

- **Checkpointing:** Add `.option("checkpointLocation", ...)` to `writeStream` for fault tolerance and at-least-once delivery guarantees.
- **Pandas UDF:** Replace the row-level UDF with a vectorised `pandas_udf` to achieve batch processing within each partition, significantly improving throughput.
- **MLflow integration:** Load model artefacts from an MLflow registry, enabling A/B testing of model versions in production.
- **Persistent sink:** Write decisions to a Delta Lake table, PostgreSQL database, or a downstream Kafka topic for consumption by a dashboard or alerting service.
- **Containerisation:** Package the entire stack in a `docker-compose.yml` for reproducible one-command deployment.
- **Dead-letter queue:** Route unparseable or prediction-failed messages to a dedicated Kafka error topic rather than silently discarding them.
- **Multi-partition scaling:** Increase Kafka topic partition count and configure multiple Spark executors to process partitions in parallel.

13.4 Final Remarks

The project demonstrates that integrating classical machine learning with modern streaming infrastructure is both feasible and practical with current open-source tooling. The architecture described here forms a solid foundation for a production-grade supply chain risk management system. With the enhancements outlined — particularly checkpointing, vectorised UDFs, and a durable output sink — this pipeline would be well-positioned for deployment in a real-world logistics or e-commerce environment where timely, data-driven order decisions translate directly to cost savings and improved customer satisfaction.